4 Experiments and Results

This chapter reports the experiments on the GazeCapture data set [1] and on the personal recording sessions. It first describes the model, the data and the evaluation protocol (Section 4.1). It then analyses the training hyperparameters (Section 4.2), the spatial distribution of the prediction error and the loss functions (Section 4.3), and the personalization sessions (Section 4.4). Section 4.5 discusses what these results mean for the rest of the thesis.

All values were re-checked against the experiment logs. During this check an error in the evaluation code was found and corrected (Section 4.1.3). All model errors in this chapter are reported with the corrected metric.

4.1 Experimental Setup

4.1.1 Model

- **Backbone:** ResNet-18 [2], pretrained on ImageNet (`ResNet18_Weights.DEFAULT`).
- **Regression head:** four fully connected layers $512 \to 512 \to 256 \to 128 \to 2$ with ReLU, dropout $p = 0.2$ after the second layer, and an output activation (default: Sigmoid).
- **Optimizer:** AdamW, weight decay 10^{-5}.
- **Default loss:** L1 loss (`nn.L1Loss`), see Section 4.3.3.
- **Early stopping:** patience of three epochs, maximum 500 epochs.

4.1.2 Data and Splits

GazeCapture contains 347 085 labelled frames of 262 subjects in the version used here. Two splits are used:

- `norm_subject`: subject-disjoint split (no person appears in both training and test data);
- `norm_random`: frame-level split, the same person can appear in training and test data.

For the hyperparameter study, 1000 training and 200 test frames are drawn *at random* from the split files (fixed seed 0, `-sample-seed 0`). For `norm_subject` the training subset contains 179 subjects and the test subset 45 subjects, with no overlap. (Earlier runs used the *first* rows of the sorted split files, i.e. only five training subjects and one test subject; these runs are no longer used for the tables.)

4.1.3 Evaluation Metric

All gaze targets are normalized to the unit square $[0, 1]^2$ (Section 4.1.5). The error is the root mean squared *Euclidean* distance between prediction $\hat{\boldsymbol{p}}_i$ and target $\boldsymbol{p}_i$, expressed as a percentage of the screen diagonal $\sqrt{2}$:

$$E_{\text{diag}} \;=\; 100 \cdot \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|_2^2}}{\sqrt{2}} \quad [\%]. \tag{4.1}$$

A value of 0 % is a perfect prediction, and 100 % is an error as large as the full screen diagonal.

4.1.4 Protocol

The hyperparameter study is run with the command

```
python exp.py -experiment all -sample-seed 0 -val-size 200.
```

One axis (for example the batch size) is changed, while all other values stay at the default configuration (Table 4.1). Each configuration is trained five times. Early stopping uses a subject-disjoint validation set of 200 frames, and the reported value is the *test* error at the epoch with the lowest validation error, so the test set is not used for model selection. The tables report the mean test error, the two-sided 98 % confidence interval (CI) of the mean (Student-t, four degrees of freedom) and the mean training time.

Reading the tables. In every hyperparameter table the selected option is marked with †. The column "p vs. †" gives the p-value of a two-sided Mann–Whitney U test between the five runs of that option and the five runs of the selected one. A rank test is used because five runs are too few to check normality and several options contain single outlier runs (e.g. one ReLU run at 46.98 %). With $5 + 5$ runs the smallest possible p-value is 0.008; it means that *every* run of one option is better than *every* run of the other. A mean is printed in **bold** only if the selected option is *significantly better* than every other option of the same table ($p < 0.05$ and lower mean).

Three properties of this protocol must be kept in mind:

Table 4.1: Default configuration of `exp.py`. Each sweep changes exactly one of these values.

Parameter	Value	Parameter	Value
Split	`norm_subject`	Trainable layers	FC only
Train / test size	1000 / 200[a]	Output activation	Sigmoid
Batch size	32	Loss	L1 (`standard`)
Learning rate	10^{-4}	Max. epochs / patience	500 / 3
Weight decay	10^{-5}	Validation set	200 frames[b]
Repeats	5		

[a] drawn at random, seed 0. [b] from one subject that is not in the training or test subset.

1. **Same data in all repeats.** The seed fixes the train, test and validation subsets, so the five repeats differ only in weight initialization, data order and dropout. The CI therefore measures training randomness, not the variation between different data subsets.
2. **Small validation set.** The 200 validation frames come from a single subject. Early stopping is therefore noisy, but the test set is never used for model selection.
3. **Training time.** For FC-only runs the head is trained on cached ResNet features. The reported times therefore do not contain the forward pass of the backbone and are not comparable with the times of the configurations that train parts of the backbone.

4.1.5 Target Normalization

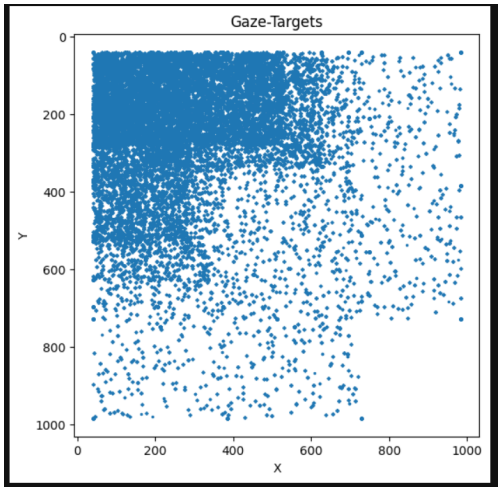
The raw gaze targets are given in device-dependent units, so their range depends on the screen size and the orientation of the device. The targets are therefore normalized with the screen width W and height H:

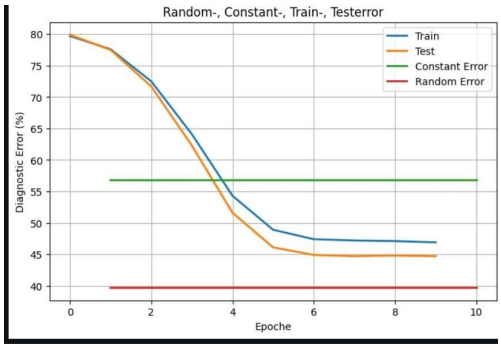
$$x_{\text{norm}} = \frac{x}{W}, \qquad y_{\text{norm}} = \frac{y}{H}. \tag{4.2}$$

Figure 4.1 shows the effect. Without normalization the error values are not comparable between devices (left column). After normalization all targets lie in $[0, 1]^2$ and Equation (4.1) is well-defined (right column). All further experiments use normalized targets.

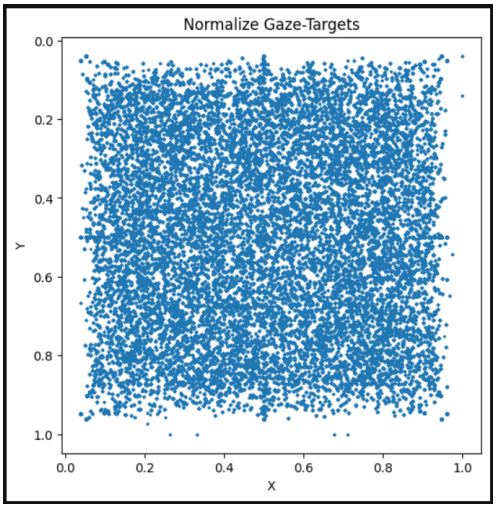
Why does the curve start at the constant and not at the random baseline? In Figure 4.1b the curves start at 26.77 % at epoch 0, i.e. *before* the first training step. One could expect the untrained model to behave like the random baseline (about 40 %). This expectation does not hold, for the following reason.

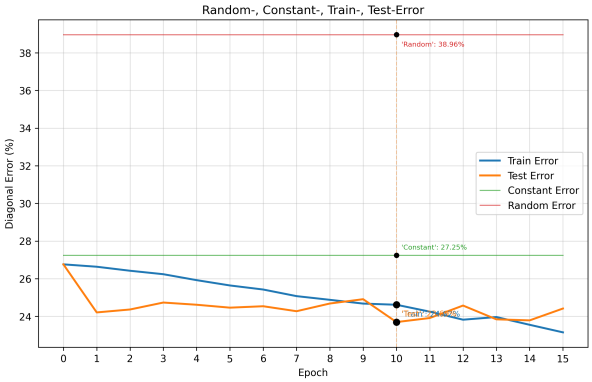
The random baseline guesses a *different*, uniformly distributed point for every image, spread over the whole screen. An untrained network does not do this. Its weights are initialized with small random values, so the last linear layer produces values close to 0 for every image,





(a) Raw targets. Bottom: learning curve of an early run on raw coordinates; the values are not percentages of a common screen diagonal and cannot be compared with the other figures.





(b) Normalized targets. Bottom: learning curve on `norm_random` (1000/200) with the metric of Equation (4.1). Epoch 0 is the untrained model. The horizontal lines are baselines of the *full* data set; see text.

Figure 4.1: Raw versus normalized gaze targets.

and the Sigmoid maps them to $\sigma(0) = 0.5$. The untrained model therefore predicts almost the *same* point, the screen centre (0.5, 0.5), for all images. This is exactly what the constant baseline does. Hence the correct expectation is:

- epoch 0 $\approx$ constant baseline of the *same* test data (a "always look at the centre" guess);
- training must bring the curve *below* this value; only then has the model learned something from the images;
- the random baseline is an upper reference for "pure guessing"; a model should never be above it.

The measured values confirm this: on the 200 test images of this run the constant baseline is 26.76 % (training mean) or 26.71 % (screen centre), which equals the start value of 26.77 %. The horizontal line in the old figure (27.25 %) was computed on the *full* data set and therefore lies slightly higher; this made the curve seem to start "below the baseline". The plotting code was corrected: the baselines are now computed on the same train/test subset as the curves (`metrics.subset_baselines`; `Compare_notebook.ipynb`, Section 4, which also re-creates this figure).

4.1.6 Baselines

Two reference predictors are used. The *constant baseline* predicts the mean target of the training data for every test image. The *random baseline* draws predictions uniformly from $[0, 1]^2$. A model is only useful if it is clearly better than the constant baseline.

Table 4.2: Baseline errors E_{diag} in %. Top: full data set (`exp.py`, bootstrap of 30 samples with 2000 frames, mean and 98 % CI). Bottom: the same random train/test subsets (seed 0) that are used for model training (no CI, one fixed subset).

Data	Constant baseline	Random baseline
Full data set (347 085 frames)	27.28 [27.15, 27.40]	39.82 [39.60, 40.04]
`norm_subject`, 1000/200	28.46	40.36
`norm_subject`, 10 000/2000	27.62	39.96
`norm_random`, 1000/200	27.66	39.83

The baselines of the earlier draft (30.99 % and 42.70 %) came from an older notebook run that is not part of the logs; they are replaced by the logged values in Table 4.2. The lower part of the table is the correct reference for all model results in Section 4.2.

4.2 Training Configuration

All results in this section come from one run (`exp_results_all.json`; `exp.py`, corrected metric, random subsets with seed 0, validation-based early stopping, five repeats per configuration). The same-subset constant baseline of the default configuration is 28.46 %.

4.2.1 Batch Size

Table 4.3: Batch size. Mean test error over five runs with 98 % CI and mean training time (FC-only head on cached features).

Batch size	Mean test error (%)	98 % CI (%)	p vs. †	Mean time (min)
32†	28.39	[28.21, 28.58]	—	0.019
64	28.35	[28.29, 28.41]	0.69	0.012
128	28.38	[28.30, 28.47]	0.84	0.009

Table 4.3 shows that the three batch sizes give the same test error: the means differ by at most 0.05 percentage points, the confidence intervals overlap, and the rank test finds no difference ($p \geq 0.69$). The training times are below one second per run, because only the small head is trained on cached features; the differences in time are therefore not relevant.

A batch size of 32 is kept as default. The experiment shows no disadvantage of this value, and there are two practical arguments for a small batch in the personalization setting: (i) a personal session contains only a few hundred training images, so a small batch gives more parameter updates per epoch, and (ii) the gradient noise of small batches is commonly associated with a mild regularizing effect. The data of this experiment do *not* prove an advantage of 32 over 64 or 128.

4.2.2 Learning Rate

Table 4.4: Learning rate. Mean test error over five runs with 98 % CI and mean training time.

Learning rate	Mean test error (%)	98 % CI (%)	p vs. †	Mean time (min)
10^{-3}	28.43	[28.33, 28.54]	0.15	0.018
10^{-4}†	28.37	[28.32, 28.42]	—	0.020
10^{-5}	28.33	[28.28, 28.38]	0.056	0.022
10^{-6}	28.37	[28.21, 28.52]	0.42	0.055

Table 4.4 shows no significant difference between the four learning rates in the five-run sweep: all means lie between 28.33 % and 28.43 %, the confidence intervals overlap, and no

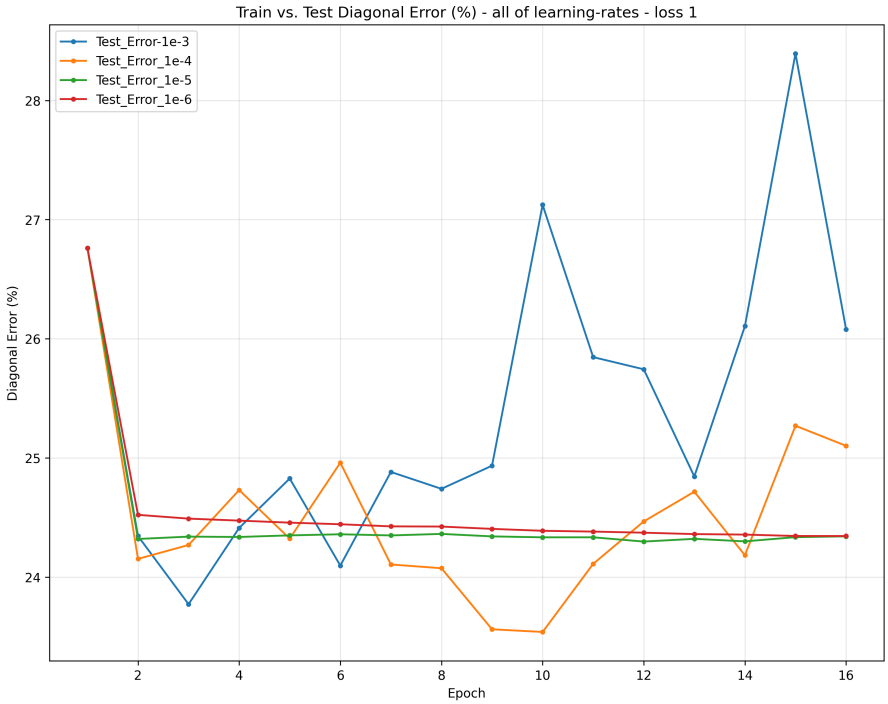


Figure 4.2: Test error curves of single training runs with four learning rates (`norm_random`, 1000/200, 15 epochs without early stopping, metric of Equation (4.1)). The first point (epoch 1 on the axis) is the untrained model.

rank test is significant ($p \geq 0.056$). The final error alone therefore cannot justify the choice. The choice of $\eta = 10^{-4}$ is based on the *convergence behaviour* in Figure 4.2:

- $\boldsymbol{\eta = 10^{-3}}$: the steps are too large. After a fast first decrease the test error oscillates strongly and grows again (peaks of about 27.1 % and 28.4 %); the run does not converge to a stable minimum.
- $\boldsymbol{\eta = 10^{-4}}$: reaches the lowest test error of all curves (about 23.6 % around epoch 10). The curve still fluctuates, but much less than with 10^{-3}, and early stopping can keep the best state.
- $\boldsymbol{\eta = 10^{-5}}$ **and** $\boldsymbol{10^{-6}}$: the curves are very smooth and converge monotonically, but they stay on a plateau of about 24.3 %. The step size is too small to leave this region within the training budget; the result is stable but clearly not the best reachable one.

The training time is *not* used as a criterion here: it is below one second for all learning rates (cached features), and the decision depends only on convergence behaviour and the reachable error. Note that Figure 4.2 shows single runs; the five-run sweep in Table 4.4 does not confirm a significant difference.

4.2.3 Trainable Layers

Three fine-tuning depths are compared (Table 4.5): only the regression head (FC only), the head plus the last residual stage (`layer4`), and the complete network.

Table 4.5: Trainable parameters of the three fine-tuning configurations (ResNet-18 with regression head, 11 603 650 parameters in total).

	FC only	FC + Layer 4	Full ResNet
Trainable parameters	427 138	8 820 866	11 603 650
Share of all parameters	3.68 %	76.02 %	100.00 %
Backbone features cacheable	yes	no	no

Table 4.6: Trainable layers. Mean test error over five runs with 98 % CI and mean training time. The FC-only time excludes the backbone forward pass (cached features).

Trainable layers	Mean test error (%)	98 % CI (%)	p vs. †	Mean time (min)
FC only†	28.36	[28.23, 28.49]	—	0.015
FC + Layer 4	29.07	[26.58, 31.55]	0.22	9.160
Full ResNet	29.39	[26.99, 31.78]	0.31	15.366

FC only has the lowest mean error and by far the narrowest interval (Table 4.6). Training more layers does not help; it makes the result *less stable*: one of the five FC + Layer 4 runs (31.72 %) and two of the five full fine-tuning runs (30.70 % and 31.18 %) end clearly above the constant baseline (28.46 %), while all FC-only runs stay between 28.28 % and 28.47 %. With only five runs the differences are not significant ($p = 0.22$ and 0.31), so the FC-only value is not printed in bold. The training time grows strongly with the number of trainable parameters: 9.2 min per run for FC + Layer 4 and 15.4 min for the full network.

FC only is selected for the personalization experiments for three reasons:

1. **No loss in accuracy, more stability:** in this experiment, training more layers does not reduce the error but increases the spread between runs.
2. **Robustness with few data:** a personal session has only a few hundred images. Updating millions of backbone parameters with so few samples risks overfitting and distorts the pretrained features; Kumar et al. [3] show that full fine-tuning can damage pretrained representations and generalize worse than training only the head.
3. **Cost:** only 3.68 % of the parameters are trained and the backbone features can be computed once and reused. This makes personalization fast and memory-efficient.

4.2.4 Output Activation

The output activation is applied after the last linear layer that produces (x, y). Sigmoid maps the output to $(0, 1)$, which matches the normalized coordinate range. The Gaussian activation $g(z) = \exp(-z^2/2\sigma^2)$ with $\sigma = 1$ also maps to $(0, 1]$ but is not monotonic. ReLU only removes negative values, and "None" leaves the linear output unchanged.

Table 4.7: Output activation. Mean test error over five runs with 98 % CI and mean training time.

Activation	Mean test error (%)	98 % CI (%)	p vs. †	Mean time (min)
Sigmoid†	**28.36**	[28.29, 28.43]	—	0.022
Gaussian	31.29	[28.44, 34.14]	0.008	0.019
ReLU	32.90	[19.69, 46.10]	0.008	0.028
None	29.39	[28.42, 30.36]	0.008	0.032

Sigmoid is the only clear winner of the hyperparameter study (Table 4.7): it has the lowest mean error and the narrowest interval, and *every* Sigmoid run is better than *every* run of the three other activations ($p = 0.008$ in all three comparisons). The value is therefore printed in bold. The other activations are also less stable: one ReLU run ends at 46.98 %, which is worse than the random baseline. A plausible reason is that ReLU outputs can become zero for all inputs ("dead" outputs), from which the gradient cannot recover. Without a bounding activation (None, ReLU) the network can also predict points outside the screen. Sigmoid is selected because it bounds the output to the valid screen range and gave the lowest and most stable error.

4.2.5 Data Set Size

Table 4.8: Data set size (train/test). Mean test error over five runs with 98 % CI, mean training time and the constant baseline of the respective test subset.

Train / test	Mean test error (%)	98 % CI (%)	p vs. †	Time (min)	Const. baseline (%)
10 000 / 2000	27.64	[27.55, 27.73]	0.008	0.167	27.62
1000 / 200†	28.39	[28.33, 28.44]	—	0.018	28.46

The larger configuration has a lower mean error (Table 4.8). However, the two configurations are evaluated on *different* test sets, and each mean is practically equal to the constant baseline of its own test set (27.64 % vs. 27.62 % and 28.39 % vs. 28.46 %). The difference of 0.75 percentage points is therefore caused by the different test data and not by more training data, even though the rank test is significant ($p = 0.008$). The small configuration (1000/200) is kept for the sweeps because it is about ten times faster and corresponds to the data volume of a personalization session.

4.2.6 Split Type

Table 4.9: Split type. Mean test error over five runs with 98 % CI and the constant baseline of the respective test subset.

Split	Mean test error (%)	98 % CI (%)	p vs. †	Constant baseline (%)
`norm_random`	27.72	[27.64, 27.80]	0.008	27.66
`norm_subject`†	28.39	[28.26, 28.53]	—	28.46

The random split gives the lower error (Table 4.9). Again, most of the difference comes from the test subsets: the constant baseline is already 0.80 percentage points lower on the `norm_random` subset, and both models are within 0.07 percentage points of their own baseline. In addition, the random split lets the same persons appear in training and test data, which makes the task easier and the estimate optimistic. The subject-disjoint split `norm_subject` is therefore used as the default: it is the only one that measures generalization to unseen persons, which is the situation before personalization.

4.3 Spatial Error Analysis and Loss Functions

The mean error of Section 4.2 hides *where* on the screen the errors occur. This section first analyses the spatial error distribution (Sections 4.3.1 and 4.3.2). The observed pattern motivated position-weighted loss functions (Section 4.3.3), which are evaluated in Section 4.3.4.

4.3.1 Screen Partition

The normalized screen $[0, 1]^2$ is described by a 3×3 reference grid with three regions:

$$\begin{aligned} \text{Centre:} &\quad (0.50,\, 0.50), \\ \text{Edges:} &\quad (0.10,\, 0.50),\ (0.50,\, 0.10),\ (0.90,\, 0.50),\ (0.50,\, 0.90), \\ \text{Corners:} &\quad (0.10,\, 0.10),\ (0.10,\, 0.90),\ (0.90,\, 0.10),\ (0.90,\, 0.90). \end{aligned}$$

A test sample with target $\boldsymbol{p}$ belongs to grid point $\boldsymbol{g}$ if $\|\boldsymbol{p} - \boldsymbol{g}\|_2 \leq r$ with $r = 0.15$. The error of a grid point is the mean Euclidean error $\|\hat{\boldsymbol{p}} - \boldsymbol{p}\|_2$ of its samples (in normalized units, i.e. a fraction of the screen side; the diagonal is $\sqrt{2} \approx 1.41$). The error of a region is the mean over all samples of its grid points. These definitions are used in all figures and tables of this section.

4.3.2 Error Heat Maps

Figure 4.3 uses the 200 test predictions of the default model (`norm_subject`, L1 loss; file `results/heatmap/base_loss1.csv`). For a cursor position $\boldsymbol{c}$ (centre, edge or corner), the mean error $\bar{e}(\boldsymbol{c})$ of all samples within radius r is computed and shown as an isotropic Gaussian

$$H(x, y) = \bar{e}(\boldsymbol{c}) \exp\left(-\frac{\|(x, y) - \boldsymbol{c}\|_2^2}{2\sigma^2}\right), \qquad \sigma = 0.20. \tag{4.3}$$

The colour therefore encodes the *size* of the local mean error, and the position of the bright spot shows *where* it was measured. All three panels use the same colour scale (0 to the largest value), so the colours can be compared directly.

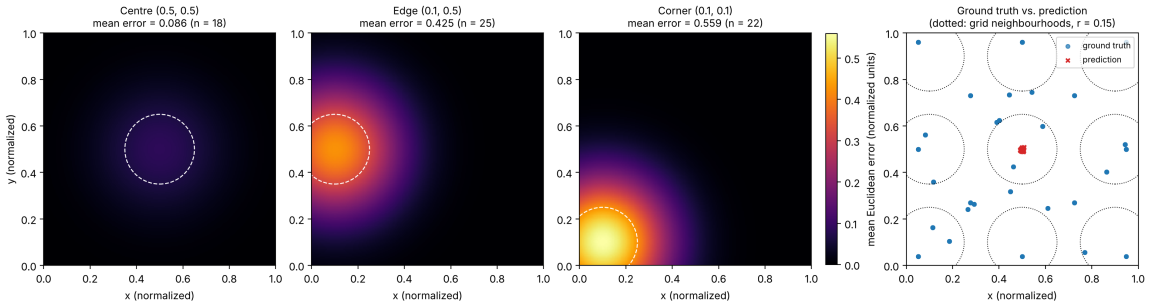


Figure 4.3: Spatial error of the default model (L1 loss, `norm_subject`, 200 test samples). Panels 1–3: Gaussian error heat maps ($r = 0.15$, $\sigma = 0.20$, common colour scale) at the centre $(0.5, 0.5)$, the left edge $(0.1, 0.5)$ and the lower left corner $(0.1, 0.1)$. The error grows from the centre (0.086) to the edge (0.425) and the corner (0.559). Panel 4: targets (blue) and predictions (red); all predictions lie close to the screen centre.

The error increases monotonically from the inside to the outside: the mean error is 0.086 in the centre region ($n = 18$), 0.432 at the edges ($n = 62$) and 0.586 in the corners ($n = 42$). Edge errors are about five times and corner errors about seven times larger than centre errors.

The fourth panel explains this pattern. All predictions lie in a very small area around $(0.5, 0.5)$: their standard deviation is about 0.004 in both coordinates, while the targets have a standard deviation of about 0.28. The model therefore predicts almost the same point for every image, and its error is simply the distance of the target to the screen centre. A constant centre predictor gives practically the same values (centre 0.085, edges 0.432, corners 0.587; overall $E_{\text{diag}} = 28.45\,\%$ versus $28.47\,\%$ for the model).

4.3.3 Loss Functions

The high edge and corner errors motivated the idea to give these regions more weight in the loss. Let $\hat{\boldsymbol{p}}_i = (\hat{x}_i, \hat{y}_i)$ be the prediction and $\boldsymbol{p}_i = (x_i, y_i)$ the target of sample i in a batch of size N.

L1 loss (`nn.L1Loss`, "standard").

$$\mathcal{L}_1 = \frac{1}{2N} \sum_{i=1}^{N} (|\hat{x}_i - x_i| + |\hat{y}_i - y_i|). \tag{4.4}$$

The gradient has a constant magnitude independent of the error size. The loss is therefore robust against outliers (for example wrong labels during target transitions), but it treats all screen positions equally.

Smooth L1 loss (`nn.SmoothL1Loss`, $\beta = 1$). For a residual u:

$$\ell_\beta(u) = \begin{cases} \dfrac{u^2}{2\beta}, & |u| < \beta, \\ |u| - \dfrac{\beta}{2}, & \text{otherwise.} \end{cases} \tag{4.5}$$

It is quadratic for small and linear for large residuals. Because all residuals in $[0, 1]^2$ are smaller than $\beta = 1$, it behaves here like a scaled squared error: the gradient becomes small when the error is small.

Sensitive L1 loss (position-weighted, `sensitive`). Each sample is weighted by its L_1 distance from the screen centre:

$$d_i = |x_i - 0.5| + |y_i - 0.5| \in [0, 1], \qquad w_i = 1 + d_i \in [1, 2], \tag{4.6}$$

$$\mathcal{L}_{\text{sens}} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \frac{|\hat{x}_i - x_i| + |\hat{y}_i - y_i|}{2}. \tag{4.7}$$

A corner sample has twice the weight of a centre sample, an edge sample about 1.4 times. This increases the gradient for outer targets and should pull the predictions away from the centre. (The earlier draft defined the weight as $w = 1/d$; this does not match the implementation in `model.py` and would be infinite at the centre.)

The "improved sensitive loss" in Table 4.11 was a further variant from an exploratory notebook. Its exact formula is not contained in the current code, so it is reported only for completeness.

4.3.4 Results of the Loss Comparison

Table 4.10 combines the global comparison from the five-run sweep with the spatial errors of the regions defined in Section 4.3.1. It replaces the two separate tables of the earlier draft (global loss table and 3×3 grid table).

Table 4.10: Comparison of L1 and Sensitive L1 loss. Left: mean test error E_{diag} over five runs (`exp.py`, 98 % CI) and mean training time. Right: mean Euclidean error per screen region (normalized units; one earlier run each, 200 test samples, $r = 0.15$; n = number of samples). Last row: constant screen-centre predictor on the same 200 test samples.

	Global (5 runs)			Spatial error (1 run)		
Loss	E_{diag} (%)	98 % CI (%)	Time (min)	Centre ($n = 18$)	Edges ($n = 62$)	Corners ($n = 42$)
L1 (standard)[†]	**28.32**	[28.23, 28.42]	0.011	0.086	0.432	0.586
Sensitive L1	28.48	[28.19, 28.78]	0.021	0.090	0.431	0.585
Constant centre	28.45[a]	—	—	0.085	0.432	0.587

[a] Evaluated on the same 200 test samples as the spatial analysis.

Table 4.11: Auxiliary experiment: spatial error of four loss functions after 15 epochs on a training set with 52 corner samples (two runs, A and B, values as logged). These numbers come from an exploratory notebook; the computation of the region values of this run is not contained in the current code, so they are not directly comparable with Table 4.10.

	Total (%)		Centre (%)		Edges (%)		Corners (%)	
Loss	A	B	A	B	A	B	A	B
L1 (`nn.L1Loss`)	24.54	24.63	17.01	16.74	28.28	28.98	34.93	35.07
Smooth L1 (`nn.SmoothL1Loss`)	23.94	24.91	15.40	16.41	29.78	30.59	34.13	35.17
Sensitive L1	25.57	25.55	19.11	18.61	29.87	29.93	33.41	34.20
Improved sensitive L1	25.53	27.75	19.20	21.86	29.17	35.03	33.77	31.58

Observations.

- Globally, L1 is slightly but significantly better than the sensitive loss: 28.32 % versus 28.48 % (Mann–Whitney $p = 0.032$, Table 4.10); the L1 value is therefore printed in bold. The sensitive loss also varies more between runs. Both values lie close to the constant baseline (28.46 %).
- Spatially, the position weighting changes the region errors by less than 0.005 normalized units. The edge and corner errors stay about five to seven times higher than the centre error.
- Both models have the same spatial profile as a constant centre predictor. The weighting increases the gradient for outer samples, but this is not enough to make the model leave the centre solution.
- In the auxiliary experiment (Table 4.11) the sensitive variants slightly reduce the corner error (Sensitive L1 by 0.9–1.5, the improved variant by 1.2–3.5 percentage points), but they increase the centre error by 1.9–5.1 percentage points and the total error in all runs. The two runs differ considerably, so these trends are not reliable.

Conclusion. The spatial analysis showed *what* the problem is: the errors are concentrated at the edges and in the corners. The position-weighted loss was the attempt to reduce these errors. It did not improve the corners and was globally even slightly but significantly worse than L1 ($p = 0.032$); in the auxiliary experiment it increased the centre error. The standard L1 loss is therefore kept. It gave the lower error, it is simpler, it has no additional hyperparameter, and its constant gradient magnitude makes it robust against label noise in the personal recordings. The analysis also shows *why* the weighting could not help: the large outer errors are not caused by a wrong weighting of screen regions but by the fact that the model barely leaves the screen centre (Section 4.5).

4.4 Personalization Sessions

In each personal session the head of the GazeCapture model is trained again on the images of one recording (FC only, Sigmoid, L1, $\eta = 10^{-4}$, batch size 32). Each session is split randomly into 70 % training, 15 % validation and 15 % test data (seed 42). Early stopping uses the validation error, and the test error is reported at the epoch with the lowest validation error. Two GazeCapture base models were used as starting points: one trained on `norm_subject` and one on `norm_random`. The column names in Table 4.12 refer to this base model; the split within a session is always random. Of the 27 recorded sessions, 18 are used (Gen 1: 01–04, Gen 2: 05–09, Gen 4: 19–27); Gen 3 (10–18) was discarded because it showed no learning effect.

Table 4.12: Personalization of sessions 01–09 with two base models. Best validation error, test error at the best validation epoch (both corrected metric, Equation (4.1)), training time, and the constant baseline of the session. Mean and 98 % CI over the nine sessions; t = training time.

	Base model `norm_subject`			Base model `norm_random`			
Session	Val. (%)	Test (%)	t (min)	Val. (%)	Test (%)	t (min)	Const. (%)
01	15.16	16.16	13.4	12.91	14.48	25.7	28.78
02	12.88	12.05	10.3	13.41	12.91	7.8	29.30
03	16.42	17.11	14.9	20.62	21.92	6.9	28.76
04	17.54	15.33	10.4	17.58	15.77	8.6	28.51
05	24.20	18.07	7.5	26.29	22.03	3.9	30.88
06	18.65	14.84	9.0	19.15	15.67	6.4	27.45
07	24.69	21.84	5.5	26.28	23.00	3.4	28.46
08	26.59	27.58	1.0	26.35	28.52	1.0	26.95
09	27.34	26.18	1.9	23.94	21.20	3.5	25.87
Mean	**20.39**	**18.79**	**8.2**	**20.72**	**19.50**	**7.5**	**28.33**
98 % CI	[15.20, 25.57]	[13.68, 23.91]	[3.6, 12.8]	[15.54, 25.91]	[14.60, 24.40]	[0.5, 14.5]	[26.93, 29.72]

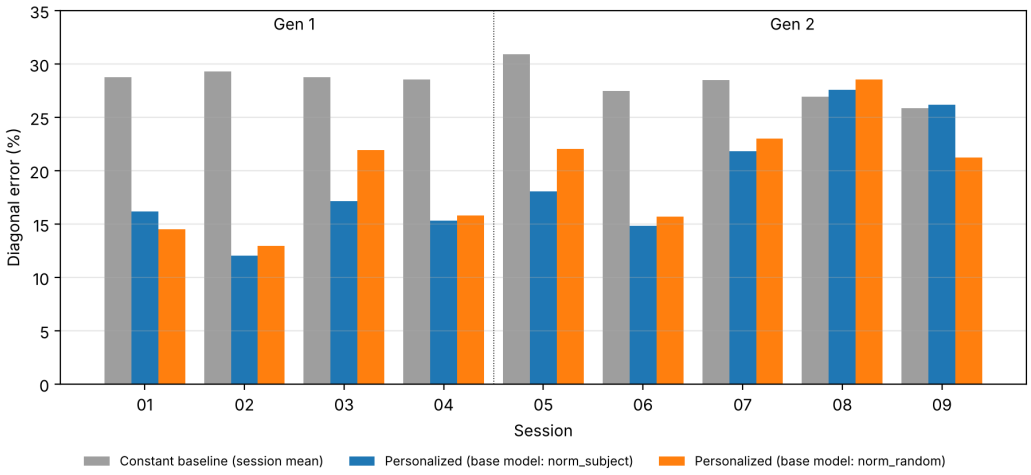


Figure 4.4: Test error of the personalized model for sessions 01–09 compared with the constant baseline of each session (corrected metric).

Personalization works. In contrast to the GazeCapture experiments, the personalized models are clearly better than the constant baseline (Table 4.12, Figure 4.4). With the `norm_subject` base model the mean test error is 18.79 % compared with 28.33 % for the constant baseline, a reduction of about one third. For all 18 sessions (Gen 1, 2 and 4) the mean test error is 19.85 % [16.87, 22.82] versus 29.85 % for the constant baseline (paired t-test, $p < 10^{-6}$). The best session (S02, bright room, no glasses) reaches 12.05 %. Sessions 08 and 09 were recorded in a very dark room; here the model is not better than the constant baseline, which shows that good illumination is a precondition.

Base model. The two base models give the same result: 18.79 % versus 19.50 % (paired t-test, $p = 0.48$). The `norm_subject` base model is used for the following experiments because it was trained without information about the test persons.

Generation 4 (WAIT/MOVE protocol). For sessions 19–27 the mean test error is 20.90 % with the original labels and 19.97 % after removing the WAIT frames (paired t-test, $p = 0.32$, not significant). Removing the WAIT frames helped strongly in two sessions (S20: -5.5, S25: -4.9 percentage points) and hurt in one (S23: $+2.7$ percentage points). The detailed analysis of this generation (glasses, background, duration, operating system) is contained in `personal_compare.ipynb` and will be presented in the next chapter.

4.5 Discussion

Main finding. On GazeCapture, all configurations of the hyperparameter study reach practically the error of a constant predictor on the same test data: the best FC-only configurations lie between 28.32 % and 28.39 %, the constant baseline is 28.46 %. The spatial analysis confirms that the predictions are concentrated at the screen centre. Consequently the hyperparameter study separates the configurations mainly by *stability*: options that make training less stable (Gaussian, ReLU, no activation, unfreezing backbone layers, the sensitive loss) are worse, but no option clearly leaves the centre solution. The choices in Section 4.2 are therefore based on stability, convergence, cost and methodological arguments, as stated in each subsection. Single notebook runs on `norm_random` (Figures 4.1b and 4.2) reached about 23.6 % versus 26.8 % for the constant baseline, so a small learning effect exists, but it is not visible in the five-run means.

Possible reasons. The new protocol already excludes two explanations that were plausible for the earlier runs: the training subset now contains 179 subjects instead of five, and the test set is no longer used for model selection; the collapse remains. The most likely remaining causes are: (i) full camera frames are used, in which the eyes cover only a small part of the

image, so the frozen ImageNet features carry little gaze information; (ii) 1000 frames from 179 subjects are only about six frames per person, too few to learn a person-independent mapping; unfreezing more layers with so few data makes training unstable instead of better (Section 4.2.3). Face or eye crops as input and larger training sets are the next steps to test.

Consequences for the code. The evaluation metric was corrected, baselines are computed on the same subset as the model, random subsets (`-sample-seed`) and a subject-disjoint validation set (`-val-size`) were added to `exp.py`, and full fine-tuning (`fc_resnet`) can be included with `-include-resnet`.

Consequence for the thesis. The personalization itself is effective: with the same frozen backbone, a head trained on a few hundred images of one person reduces the error by about one third compared with the constant baseline. The GazeCapture base model mainly provides a stable initialization. This supports the central idea of this work, data-efficient personalization of a pretrained model, while it shows that the generic (person-independent) model is not yet a strong baseline.

4.6 Summary of the Selected Configuration

Table 4.13: Selected configuration and the main reason for each choice.

Parameter	Choice	Reason
Targets	normalized $[0, 1]^2$	device-independent, metric well-defined (Section 4.1.5)
Batch size	32	no significant difference; more updates per epoch on small personal data (Section 4.2.1)
Learning rate	10^{-4}	no significant difference in the sweep; lowest reachable error in the curves, 10^{-3} unstable, $\leq 10^{-5}$ stuck on a plateau (Section 4.2.2)
Trainable layers	FC only	lowest and most stable error, 3.68 % of the parameters, protects pretrained features (Section 4.2.3)
Activation	Sigmoid	**significantly** better than all alternatives ($p = 0.008$), bounded to the screen (Section 4.2.4)
Data set size	1000/200	gain of the larger set only due to a different test set; ten times faster (Section 4.2.5)
Split	`norm_subject`	only split that measures generalization to unseen persons (Section 4.2.6)
Loss	L1	**significantly** better than the sensitive loss ($p = 0.032$); robust, no extra hyperparameter (Section 4.3.4)

References

[1] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik, and Antonio Torralba. Eye tracking for everyone. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2176–2184, 2016. doi: 10.1109/CVPR.2016.239.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[3] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.